claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_ae3ec722-12a\agent-a894acbad853722a4.jsonl`
- SHA-256: `d91f219cc588a8cb627c1a887f49771198a698639e2cdb1a0694e9766108b532`
- Source modified: `2026-07-06T07:57:10+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:52:24.498Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the
destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py,
tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:
- db.py: _migration_004_destinations_and_routing (backfills destinations from filter_config_deprecated,
  routing_rules per source, delivery_attempts from videos_deprecated.forwarded=1); repos upsert_destination,
  get_destination, upsert_delivery_attempt, record_delivery (UPSERT with attempt_count increment), get_delivery_attempt.
- delivery.py: forward_to_target/send_link_message/deliver gain an optional destination_id kwarg; when set,
  record_delivery writes a DeliveryAttempt; when None (all current live callers) falls back to the mark_forwarded no-op.
- models.py: Destination, RoutingRule, DeliveryStatus, DeliveryAttempt.
Note: routing is NOT yet wired into the live path (telethon_client/manager still call deliver() with destination_id=None);
destination_id is currently exercised only by tests. So record_delivery issues are LATENT until Track D, but ship now.

Ground rules:
- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. Build schema via db._apply_schema(conn) on aiosqlite ":memory:";
  or db.init_db(path) on a temp file seeded with a legacy v1 schema (users/channels/videos/filter_config)
  to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns
  duration_s,width,height,size_bytes,mime_type or migration 002's column-copy fails (that's a test-harness
  requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): ruff check clean; pytest 451 passed @ 95.75%
  (floor 80%); ruff format --check flags commands.py ONLY because this branch predates the merged #36 format
  fix (commands.py is NOT in this PR) ? do NOT report that as a #37 finding.

```
- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from
LATENT
  (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding.
Rank severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug,
narrow trigger),
  medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic
triggers.

DIMENSION: migration 004 SQL correctness & backfill completeness (db.py:667-760 approx).
Investigate:
- SEED to verify/refute: an account with sources + a forwarded video but NO filter_config row
gets 0 destinations,
  0 routing_rules, 0 delivery_attempts (forwarded delivery history not backfilled, no delivery
path). Bounded
  because bootstrap.py:75 always persists a default filter_config ? assess reachability &
severity.
- The delivery_attempts backfill JOINs destinations d ON d.account_id = vd.user_id (unqualified)
? if an account
  had >1 destination, does it produce a cartesian duplicate per video? (backfill makes 1, but is
it robust?)
- The routing_rules backfill WHERE d.target = (subquery json_extract delivery_target) ?
correctness across
  'me' default, '@channel', numeric chat id; any account with multiple sources; NULL/malformed
config_json.
- Idempotency gate 'COUNT(destinations)>0 return' vs the user_version ladder: redundant? fragile
to a partial
  run (destinations inserted, then crash before routing/attempts) skipping the rest on retry?
Are 004's inserts
  all DML (roll back atomically on crash)?
- kind CASE ('me'->saved else channel): numeric chat ids classified 'channel' not 'chat' ? ADR
says 'saved'|'channel'|'chat'.
- is_output=0 for the backfilled delivery destination vs ADR 0009 'distinguishes output
channels'.
Return findings per schema.
```

2. assistant (2026-07-06T07:52:29.020Z)

I'll start by reading the key files to understand migration 004 and the related code.

3. user (2026-07-06T07:52:30.523Z)

```
600                 'tg:fp:' || size_bytes || ':' || duration_s || ':'
601                 || height || ':' || width
602             WHERE document_id IS NULL
603               AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
604               AND height IS NOT NULL AND width IS NOT NULL
605               AND dedup_key LIKE 'tg:msg:%'
606               AND id IN (
607                 SELECT MIN(id) FROM content_items
608                 WHERE document_id IS NULL
609                   AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
610                   AND height IS NOT NULL AND width IS NOT NULL
611                   AND dedup_key LIKE 'tg:msg:%'
612                 GROUP BY account_id, size_bytes, duration_s, height, width
613               )
614             """
615         )
616
617         # 3. Backfill policies from filter_config: one 'default' policy per account
618         #    (Option 2A). is_default=1 so the legacy load_filter_config shim reads
619         #    this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620         await conn.execute(
621             """
622             INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
```

```
                                        created_at, updated_at)
            SELECT user_id, 'default', config_json, 1,
                   strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
                   strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
            FROM filter_config
            """
        )

        # 4. Create the policies default-partial index (sig after backfill so the
        #    UNIQUE constraint only applies once rows are populated). On fresh DBs
        #    _SCHEMA creates it; on migrated DBs this is the first time.
        await conn.execute(
            "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
            "ON policies(account_id) WHERE is_default = 1"
        )

        # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
        #    produce a content_items row. A mismatch signals fingerprint collisions
        #    where two distinct videos shared (size,dur,h,w) ? the losers stay at
        #    tg:msg: keys and are still present. Log a warning so the operator can
        #    investigate; the data is preserved in *_deprecated tables.
        async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
            vc = (await cur.fetchone())[0]
        async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
            cic = (await cur.fetchone())[0]
        if vc != cic:
            import logging

            _logger = logging.getLogger(__name__)
            _logger.warning(
                "migration_003: %d videos rows produced %d content_items rows "
                "(%d gap). Colliding items remain in videos_deprecated.",
                vc,
                cic,
                vc - cic,
            )

        # 6. Rename (don't drop) the legacy tables ? kept for one release as a
        #    rollback safety net (ADR 0010 pattern).
        await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
        await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
        await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")


    @_migration(4)
    async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
        """Backfill default destinations and delivery state (ADR 0011, Track C).

        Creates a default destination row for each account that has a
        ``filter_config_deprecated.delivery_target`` value, then creates a default
        routing rule (source ? default policy ? destination) for each source.
        Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
        table (one ``delivered`` attempt per previously-forwarded video to the
        account's default destination).

        Idempotent: skips accounts whose default destination already exists.
        """
        # Gate: if destinations already have data, this migration has run.
        async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
            if (await cur.fetchone())[0] > 0:
                return

        if not await _table_exists(conn, "filter_config_deprecated"):
            return
```

```
        # 1. Backfill default destinations from filter_config_deprecated.
        #    Parse the JSON delivery_target, defaulting to "me".
        await conn.execute(
            """
            INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
            SELECT user_id, 'telegram',
                   COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
                   CASE
                     WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
                       THEN 'saved'
                     ELSE 'channel'
                   END,
                   0,
                   strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
            FROM filter_config_deprecated
            """
        )

        # 2. Create a default routing rule per source (source ? default policy ? default
destination).
        await conn.execute(
            """
            INSERT OR IGNORE INTO routing_rules
                (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
            SELECT s.account_id, s.id, p.id, d.id, 1, 0,
                   strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
            FROM sources s
            JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
            JOIN destinations d ON d.account_id = s.account_id
            WHERE d.target = (
                SELECT COALESCE(
                    json_extract(fcd.config_json, '$.delivery_target'), 'me'
                )
                FROM filter_config_deprecated fcd
                WHERE fcd.user_id = s.account_id
                LIMIT 1
            )
            """
        )

        # 3. Backfill delivery_attempts from previously-forwarded videos.
        await conn.execute(
            """
            INSERT OR IGNORE INTO delivery_attempts
                (content_item_id, destination_id, route_id, status, attempt_count,
                 created_at, delivered_at)
            SELECT ci.id, d.id, rr.id, 'delivered', 1,
                   vd.seen_at, vd.seen_at
            FROM videos_deprecated vd
            JOIN content_items ci
              ON ci.account_id = vd.user_id
             AND ci.provider = 'telegram'
             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
            JOIN sources sc
              ON sc.id = ci.source_id
             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
            JOIN destinations d ON d.account_id = vd.user_id
            JOIN routing_rules rr
              ON rr.account_id = vd.user_id
             AND rr.source_id = sc.id
             AND rr.destination_id = d.id
            WHERE vd.forwarded = 1
```

```
            """
        )


    async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
        """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
        async with conn.execute(
            "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
        ) as cur:
            return await cur.fetchone() is not None


    # --- helpers ---------------------------------------------------------------


    def _now() -> datetime:
        """Timezone-aware UTC 'now', used for policy timestamps."""
        return datetime.now(UTC)


    def _safe_int_cast(value: object, field_name: str) -> int:
        """Cast a provider_source_id / provider_message_id / document_id to int.

        All existing data is Telegram ? these ids are numeric strings. Non-Telegram
        providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
        shapes (which carry Telegram-specific int ids); this function raises a clear
        ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
        (ADR 0011 provider-neutrality guard ? review finding #5).
        """
        try:
            return int(value)  # type: ignore[arg-type]
        except (ValueError, TypeError) as exc:
            raise ValueError(
                f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
"
                f"types are Telegram-specific; use ContentItem/Source directly for "
                f"non-Telegram providers (ADR 0011)."
            ) from exc


    def _row_to_account(row: aiosqlite.Row) -> Account:
        return Account(
            id=row["id"],
            telegram_id=row["telegram_id"],
            provider=row["provider"],
            display_name=row["display_name"],
            created_at=row["created_at"],  # stored as ISO string; pydantic parses
        )


    def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
        return TelegramSession(
            account_id=row["account_id"],
            phone=row["phone"],
            session_path=row["session_path"],
            enabled=bool(row["enabled"]),
            created_at=row["created_at"],
            last_login_at=row["last_login_at"],
        )


    def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
        """Project an (account, optional session) pair onto the legacy ``User`` view.

        Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
```

```
813         shims so pre-ADR-0010 callers and tests keep working. A bot-only account
814         (no session) maps to a ``User`` with ``enabled=False`` so the legacy
815         "list enabled users" semantics exclude it ? matching the structural rule
816         that bot-only users are not runnable Telethon clients.
817         """
818         if session is None:
819             return User(
```

4. user (2026-07-06T07:52:30.983Z)

```
1       # ADR 0011: Provider-neutral domain schema (Track B)
2
3       - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4       - **Date**: 2026-07-06
5       - **Decision owner**: project owner
6       - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7       - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8       - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10      ## Context
11
12      ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13      (MTProto credential). That unblocked the *identity* dimension. This ADR
14      tackles the **content dimension**: how to represent sources, content items,
15      policies, destinations, routes, and delivery attempts in a provider-neutral
16      way, so that the system can ingest from Telegram today and from WhatsApp
17      (or any future provider) later without another schema rewrite.
18
19      ### What we have today (post-Track-A)
20
21      | Table | Meaning | Provider-coupling |
22      |---|---|---|
23      | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24      | `telegram_sessions` | Optional MTProto session (ADR 0010) | Telegram-specific by
design |
25      | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26      | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27      | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29      The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30      - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31      - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32      - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
33
34      ### What we need
35
36      The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37      `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38      defines the schema for each, the **two open design questions** that gate the
39      schema, and the migration path from the current tables.
40
41      ## The two decisions I need from you
42
43      ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45      The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46      concept). A provider-neutral `ContentItem` needs a dedup story that works
47      across providers. Three options:
48
49      #### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
```

Keep ADR 0006's three tiers but generalize them behind a single
`dedup_key` column on `content_items`:

```
content_items.dedup_key TEXT NOT NULL
```

The *value* of `dedup_key` is computed by an explicit decision tree at
ingestion time, provider by provider. The tree is ordered strongest-key
first and each branch is **guarded so partial metadata can never collapse
to a degenerate key** (e.g. `tg:fp:0:0:0:0`):

- **Telegram:**
  1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
  2. Else if **all four** fingerprint fields (`size_bytes`, `duration_s`,
     `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
  3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
     (Tier 0; the source/channel id is required because Telegram
     `message_id` is only unique within a channel, not across sources).
- **Future WhatsApp:** `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
  ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
  exposes a content hash.

The guard in step 2 (all four fields required) is what prevents the
partial-metadata collision the first draft of this ADR allowed. The pure
`compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
size_bytes, duration_s, height, width)` function that implements this tree
gets its own unit tests in the implementation PR, including cases for
partial/null metadata.

A **partial unique index** on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
replaces the current Tier-1 index and enforces per-account dedup at the DB
level, regardless of provider.

Why recommended:
- Zero new bandwidth ? all keys are computed from metadata already extracted
  (honors ADR 0003's no-download principle).
- The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
  structurally impossible.
- Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
  existing `document_id` / metadata columns during the migration.
- Supersedes ADR 0006 cleanly: the three tiers become *one* column with
  provider-specific derivation rules, documented in the migration.

Costs:
- The migration must compute `dedup_key` for every existing `videos` row
  (cheap ? pure SQL string building, no I/O).
- The `dedup_key` derivation logic lives in code (a pure function per
  provider), not in the schema. This is a feature, but it means the
  derivation rules need their own unit tests.

Option 1B: content hash (SHA-256 of downloaded bytes)

`dedup_key = sha256(downloaded_content)`. Cryptographically strong,
provider-agnostic, catches re-encodes that 1A misses.

Rejected ? violates ADR 0003 (metadata-first, no download in the hot
path). Downloading every incoming video to hash it would multiply bandwidth
and latency by orders of magnitude. Could be added later as an optional
deep-inspect stage (deferred to Track F per #31).

Option 1C: provider-assigned only (drop Tier 2)

`dedup_key = "tg:doc:{document_id}"` always; no metadata fingerprint

fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new `document.id` ? a real and common case (someone re-encodes and re-uploads the same video to a different channel).

Recommendation: 1A. It preserves the dedup quality of ADR 0006, generalizes cleanly, and honors the no-download constraint.

Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?

The codebase already has `filter_config.config_json` (one row per account, holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The umbrella introduces a `Policy` concept. They overlap. Two options:

Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)

`policies` is a new table; `filter_config` is migrated into it. One account can own multiple policies (enables per-route filter overrides from ADR 0009), with a `DEFAULT policy` per account for the no-rules-configured case.

```
policies (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id   INTEGER NOT NULL REFERENCES accounts(id),
    name         TEXT NOT NULL,                    -- e.g. "default", "reggae-route"
    config_json  TEXT NOT NULL,                    -- the FilterConfig blob (Phase 1)
    is_default   INTEGER NOT NULL DEFAULT 0,       -- one per account (UNIQUE partial idx)
    created_at   TEXT NOT NULL,
    updated_at   TEXT NOT NULL,
    UNIQUE (account_id, name)
);
```

Phase 1 (this track): `config_json` holds exactly today's `FilterConfig` JSON ? the rename is purely structural. **Phase 2** (Track F, deferred): `config_json` grows to include topic preferences and security rules, but that's behind the same column so no further migration.

Migration: for each `filter_config` row, insert a `policies` row with `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop `filter_config` after the migration (keep `filter_config_deprecated` for one release, mirroring ADR 0010's safety-net pattern).

Why recommended:
- `Policy` becomes the single home for "what to filter/enrich" ? no second overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
- Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`) without a second migration later.
- The existing `db.load_filter_config(account_id)` / `save_filter_config` become thin shims over `policies WHERE account_id=? AND is_default=1`, mirroring the Track-A `User` shim pattern ? backwards-compatible.

Costs:
- One more table migration. `filter_config` ? `policies` is a rename+extend.
- The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape); `Policy` is the *container* (name, owner, default flag) around it. Two models, not one ? but they compose cleanly.

Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer

Don't touch `filter_config`. `policies` references a `filter_config` row plus adds enrichment/security stages as separate columns.

Rejected ? creates two overlapping concepts (`filter_config` for the v1 rules, `policies` for "everything else"), which is exactly the conflation the umbrella exists to dissolve. We'd be back here in a year merging them.

Recommendation: 2A. One concept, one table, phased growth via the same JSON column.

Proposed schema (assuming 1A + 2A)

`sources` (generalizes `channels`)

A configured place we ingest from. Maps 1:1 to today's per-account `channels` rows for Telegram; future providers add their own source kinds.

```
sources (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id        INTEGER NOT NULL REFERENCES accounts(id),
    provider          TEXT NOT NULL DEFAULT 'telegram',
    provider_source_id TEXT NOT NULL,        -- Telegram channel_id; future: WA group id
    title             TEXT,
    kind              TEXT NOT NULL DEFAULT 'channel',  -- 'channel' | 'group' | 'feed'
    watched           INTEGER NOT NULL DEFAULT 0,       -- ADR 0009 selective scanning
    seen_first_at     TEXT NOT NULL,
    last_seen_at      TEXT,
    UNIQUE (account_id, provider, provider_source_id)
);
```

`channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`; `channels.channel_id` maps to `sources.provider_source_id`. The `watched` flag (ADR 0009, deferred to Track E) is included here so Track E doesn't need another migration.

`content_items` (generalizes `videos`)

A normalized item discovered from any source. Phase 1 stores only videos (matches today's `videos`); the columns are provider-neutral but the Phase-1 migration only populates video fields.

```
content_items (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id        INTEGER NOT NULL REFERENCES accounts(id),
    source_id         INTEGER NOT NULL REFERENCES sources(id),
    provider          TEXT NOT NULL DEFAULT 'telegram',
    provider_message_id TEXT NOT NULL,       -- Telegram message_id; future: WA message
id
    content_type      TEXT NOT NULL DEFAULT 'video',  -- 'video' | 'link' | 'article' |
...
    text              TEXT,                  -- caption / post text (nullable)
    link_url          TEXT,                  -- for link/article posts
    media_ref         TEXT,                  -- provider media reference (file path,
blob id)
    document_id       TEXT,                  -- Telegram document.id (kept for TG-
specific lookups)
    duration_s        INTEGER,
    width             INTEGER,
    height            INTEGER,
    size_bytes        INTEGER,
    mime_type         TEXT,
    dedup_key         TEXT NOT NULL,         -- ADR 0011 Question 1 (Option 1A)
    matched           INTEGER NOT NULL DEFAULT 0,
```

```
        seen_at           TEXT NOT NULL,
        -- Tier-0 equivalent: provider_message_id is only unique within a source
        -- (a Telegram message_id is per-channel), so source_id is required to
        -- avoid false collisions across sources for the same account.
        UNIQUE (account_id, source_id, provider_message_id),
        UNIQUE (account_id, dedup_key)                        -- Tier-1/2 equivalent (Option
1A)
    );
    ```

    Note on `forwarded`: dropped from `content_items`. Per-destination
    delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
    (before Track C lands), `content_items.matched` alone is sufficient ?
    delivery still goes to the single `delivery_target`, and the boolean
    `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
    so the v1 delivery path keeps working. Track C removes it.

    ### `policies` (generalizes `filter_config`) ? Option 2A

    As specified in Question 2 above.

    ### `destinations` (new ? Track C will populate)

    Defined here so the schema is complete, but **Track C (#28) implements the
    delivery layer**. Phase-1 presence is the table only (no delivery logic).

    ```
    destinations (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        account_id  INTEGER NOT NULL REFERENCES accounts(id),
        provider    TEXT NOT NULL DEFAULT 'telegram',
        target      TEXT NOT NULL,           -- 'me', '@username', chat id (ADR 0005)
        kind        TEXT NOT NULL DEFAULT 'saved',  -- 'saved' | 'channel' | 'chat'
        title       TEXT,
        is_output   INTEGER NOT NULL DEFAULT 0,      -- ADR 0009: distinguishes output
channels
        created_at  TEXT NOT NULL,
        UNIQUE (account_id, provider, target)
    );
    ```

    ### `routing_rules` (new ? Track C will populate)

    Per ADR 0009 (canonical name locked in issue #25).

    ```
    routing_rules (
        id             INTEGER PRIMARY KEY AUTOINCREMENT,
        account_id     INTEGER NOT NULL REFERENCES accounts(id),
        source_id      INTEGER NOT NULL REFERENCES sources(id),
        policy_id      INTEGER,               -- nullable ? use account's default policy
        destination_id INTEGER NOT NULL REFERENCES destinations(id),
        enabled        INTEGER NOT NULL DEFAULT 1,
        priority       INTEGER NOT NULL DEFAULT 0,
        created_at     TEXT NOT NULL,
        UNIQUE (account_id, source_id, destination_id)
    );
    ```

    ### `delivery_attempts` (new ? Track C will populate)

    Per #28's idempotency requirement (unique index on
    `(content_item_id, destination_id)`).

    ```
```

```
    delivery_attempts (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
        content_item_id INTEGER NOT NULL REFERENCES content_items(id),
        destination_id  INTEGER NOT NULL REFERENCES destinations(id),
        route_id        INTEGER REFERENCES routing_rules(id),
        status          TEXT NOT NULL DEFAULT 'pending',  --
'pending'|'delivered'|'failed'|'retrying'
        attempt_count   INTEGER NOT NULL DEFAULT 0,
        last_error      TEXT,
        created_at      TEXT NOT NULL,
        delivered_at    TEXT,
        next_retry_at   TEXT,
        UNIQUE (content_item_id, destination_id)         -- ADR #28 idempotency
    );
    ```

    ## Migration strategy (integer-preserving, phased)

    This track lands **`sources`, `content_items`, and `policies`** only.
    `destinations`, `routing_rules`, `delivery_attempts` are defined in the
    schema (so the ADR is complete) but their tables are created empty and
    populated by Track C. This keeps Track B focused and reviewable.

    ### Migration `_migration_003_domain_schema` (idempotent)

    1. **Create** `sources`, `content_items`, `policies`, `destinations`,
       `routing_rules`, `delivery_attempts` (all `IF NOT EXISTS`).
    2. **Backfill `sources`** from `channels`:
       `provider='telegram'`, `provider_source_id=channel_id`, `watched=1`
       (ADR 0009 backfill: already-seen channels stay watched).
    3. **Backfill `content_items`** from `videos`, computing `dedup_key` via the
       explicit decision tree from Option 1A (every branch reachable, no
       `COALESCE(...,0)` collapsing partial metadata to a degenerate key):
       - If `document_id IS NOT NULL`:
         `dedup_key = 'tg:doc:' || document_id`.
       - Else if `size_bytes IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
AND width IS NOT NULL`:
         `dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width`.
       - Else (Tier-0 fallback; partial or missing media metadata):
         `dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id`,
         where `source_id` is the joined `sources.id` for the row's legacy
         `channel_id` and `provider_message_id` is the legacy `message_id`.

       The implementation PR must include migration tests that seed rows hitting
       each branch (including all-fingerprint-fields-null) and assert no two
       rows collapse to the same `dedup_key`.
    4. **Backfill `policies`** from `filter_config`: one row per account,
       `name='default'`, `is_default=1`, `config_json` copied verbatim.
    5. **Rename** (don't drop) `channels` ? `channels_deprecated`,
       `videos` ? `videos_deprecated`, `filter_config` ? `filter_config_deprecated`.
       Kept for one release as a rollback safety net (ADR 0010 pattern).

    The legacy `videos`/`channels`/`filter_config` tables and their
    `db.load_filter_config` / `upsert_channel` / `insert_video` repos stay
    working as shims over the new tables (same pattern as the Track-A `User`
    shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
    `commands.py` keep working without a flag day.

    ## Consequences

    Positive
    - Provider-neutral schema: `sources`, `content_items`, `policies`,
      `destinations`, `routing_rules`, `delivery_attempts` all carry a
      `provider` column or provider-prefixed keys. Adding WhatsApp later is a
```
```

```
364       new `provider='whatsapp'` value, not a new schema.
365     - `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
366       quality and without downloading anything.
367     - `Policy` (Option 2A) gives a single home for filter + future enrichment
368       config, enabling ADR 0009's per-route overrides with no second migration.
369     - Backwards-compatible via shims, mirroring the Track-A pattern that
370       already passed review.
371
372     **Negative**
373     - Six new tables; the schema grows significantly. Each is small and
374       single-purpose, but the surface area is larger.
375     - The dedup-key derivation logic is in code, not the schema ? needs its
376       own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377     - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378       `text`) that aren't populated until later tracks. This is intentional
379       (avoid future migrations) but means some columns are NULL until then.
380
381     ## Alternatives considered
382
383     - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384       providers in code.** Rejected ? same conflation argument as ADR 0010.
385       Telegram-specific ids in column names (`channel_id`, `message_id`,
386       `document_id`) would leak into every future provider's code path.
387     - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388       already the largest track. Splitting schema (Track B) from delivery
389       logic (Track C) keeps each PR reviewable. The empty `destinations` /
390       `routing_rules` / `delivery_attempts` tables in this track are the
391       contract Track C codes against.
392     - **`content_items` as a view over `videos` (no migration).** Rejected ?
393       views don't give us the `provider` column or the `dedup_key` index, and
394       SQLite view support for writes is limited. A real table is needed.
395
396     ## Open questions for the decision owner
397
398     1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399        rejected), or 1C (provider-assigned only).
400     2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401        or 2B (wrap, rejected).
402     3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403        / `text` as NULL until later tracks populate them? Recommendation: yes
404        (avoids a future `ALTER TABLE`).
405     4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406        Recommendation: yes.
407
408     ## References
409
410     - ADR 0003 ? metadata-first probing (the no-download constraint that rules
411       out Option 1B's content hashing).
412     - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413       `routing_rules`).
414     - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415       Option 1A).
416     - ADR 0009 ? selective scanning + multiplexed routing (depends on
417       `sources.watched` and `routing_rules`, both defined here).
418     - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419       at `accounts.id`).
420     - Issue #27 ? Track B (this ADR's tracking issue).
421     - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422
```

5. assistant (2026-07-06T07:52:33.320Z)

Let me read the models, the delivery code, and the schema for the tables involved.

6. user (2026-07-06T07:52:33.872Z)

```
"""Pydantic data models for users, channels, videos, and filter config.

These models are the in-memory representation used by the repository layer
(``db.py``) and the filter engine (``filters/``). They are deliberately
decoupled from Telethon's own types so the filter engine stays pure and
unit-testable without a live Telegram connection.
"""

from __future__ import annotations

from datetime import UTC, datetime
from enum import Enum

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator


def _utcnow() -> datetime:
    """Timezone-aware UTC 'now', used as a default factory."""
    return datetime.now(UTC)


class DeliveryMode(str, Enum):
    """How a matched video is delivered to the user.

    - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
    - ``LINK``: send a text message with a t.me link + metadata summary.
    - ``BOTH``: forward the video AND send the link message.
    """

    FORWARD = "forward"
    LINK = "link"
    BOTH = "both"


class Account(BaseModel):
    """A product identity ? the owner of config, channels, routes, and content.

    Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
    is the bridge used by the Bot API front-door (``from.id``) and by the
    legacy userbot path; it is nullable so future providers that don't have a
    Telegram id (e.g. WhatsApp) can still own an ``Account``.
    """

    model_config = ConfigDict(frozen=True)

    id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
    telegram_id: int | None = Field(
        default=None,
        description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
    )
    provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
    display_name: str | None = None
    created_at: datetime = Field(default_factory=_utcnow)


class TelegramSession(BaseModel):
    """An optional MTProto (Telethon) session linked to an :class:`Account`.

    A bot-only user has an ``Account`` row but **no** ``TelegramSession`` row;
    that is what makes them structurally invisible to ``manager.run``
    (ADR 0010). ``session_path`` is data-driven so the file-location
    convention is explicit rather than derived from the PK at runtime.
```

```
63          """
64
65          model_config = ConfigDict(frozen=True)
66
67          account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68          phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69          session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70          enabled: bool = Field(
71              default=True,
72              description="If False, the manager skips this session (not the account)",
73          )
74          created_at: datetime = Field(default_factory=_utcnow)
75          last_login_at: datetime | None = None
76
77
78      class User(BaseModel):
79          """Deprecated: legacy v1 model conflating identity + MTProto session.
80
81          Retained as a backwards-compatible view over the new ``accounts`` +
82          ``telegram_sessions`` tables (ADR 0010). New code should use
83          :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84          maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85          and tests keep working without a flag day.
86          """
87
88          model_config = ConfigDict(frozen=True)
89
90          id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91          phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92          enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93          created_at: datetime = Field(default_factory=_utcnow)
94
95
96      class Channel(BaseModel):
97          """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99          Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) ----------------------
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
```

```
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169         text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
```

```
183         seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) ---------------
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinatinons (backfilled from filter_config.delivery_target), writes
215     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216     # enforces the source/destination invariant (a channel is either source or
217     # destination, never both).
218
219
220     class Destination(BaseModel):
221         """A delivery target ? where matched content is sent (ADR 0011, Track C).
222
223         Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
224         can have multiple named output channels. Backfilled from the existing
225         per-account ``delivery_target``; the default destination is the target used
226         when no explicit routing rule matches.
227         """
228
229         model_config = ConfigDict(frozen=True)
230
231         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
232         account_id: int = Field(description="FK ? accounts.id")
233         provider: str = Field(default="telegram", description="Provider namespace")
234         target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
235         kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
236         title: str | None = None
237         is_output: bool = Field(
238             default=False,
239             description="ADR 0009: distinguishes output channels from ingest sources",
240         )
241         created_at: datetime = Field(default_factory=_utcnow)
242
243
244     class RoutingRule(BaseModel):
245         """A rule that maps a source through a policy to a destination (ADR 0009).
246
```

```
        Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
        ``policy_id`` is NULL, the account's default policy is used. When multiple
        rules match, the highest ``priority`` wins.
        """

        model_config = ConfigDict(frozen=True)

        id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
        account_id: int = Field(description="FK ? accounts.id")
        source_id: int = Field(description="FK ? sources.id")
        policy_id: int | None = Field(
            default=None,
            description="FK ? policies.id; NULL means use account default policy",
        )
        destination_id: int = Field(description="FK ? destinations.id")
        enabled: bool = Field(default=True)
        priority: int = Field(default=0, description="Higher wins when multiple rules
match")
        created_at: datetime = Field(default_factory=_utcnow)


    class DeliveryStatus(str, Enum):
        """Per-destination delivery state, replacing the legacy binary ``forwarded``."""

        PENDING = "pending"
        DELIVERED = "delivered"
        FAILED = "failed"
        RETRYING = "retrying"


    class DeliveryAttempt(BaseModel):
        """Per-destination delivery record ? idempotent via UNIQUE index.

        Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
        ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
        incremented and ``status``/``last_error`` are updated.
        """

        model_config = ConfigDict(frozen=True)

        id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
        content_item_id: int = Field(description="FK ? content_items.id")
        destination_id: int = Field(description="FK ? destinations.id")
        route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
        status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
        attempt_count: int = Field(default=0, description="Incremented on each retry")
        last_error: str | None = None
        created_at: datetime = Field(default_factory=_utcnow)
        delivered_at: datetime | None = None
        next_retry_at: datetime | None = None


    class FilterConfig(BaseModel):
        """Per-user filter configuration.

        v1 rules: duration bounds, minimum height (resolution), and max file size.
        Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
        new field later is backwards-compatible.
        """

        # Duration bounds in seconds. None means unbounded on that side.
        min_duration_s: int | None = Field(default=5, ge=0)
        max_duration_s: int | None = Field(default=7200, ge=0)

```

```
        # Resolution: a video passes if its height >= target_height.
        # None disables the resolution rule.
        target_height: int | None = Field(default=1080, ge=1)

        # Max file size in mebibytes. None disables the size rule.
        max_size_mb: int | None = Field(default=2048, ge=1)

        # Content-type exclusions. GIFs and round videos are excluded by default
        # (their duration/dimensions are usually incidental to real video filters);
        # silent videos are off by default since some legit clips have no audio.
        # Existing users' stored configs are unaffected ? the DB JSON blob overrides
        # these defaults, so this only changes the behaviour for fresh installs.
        exclude_gif: bool = Field(
            default=True,
            description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
        )
        exclude_round: bool = Field(
            default=True,
            description="If True, exclude circular video messages (round_message=True)",
        )
        exclude_nosound: bool = Field(
            default=False,
            description="If True, exclude videos without an audio track (nosound=True)",
        )

        # How matched videos are delivered: forward the video, send a link, or both.
        # Stored in the same JSON blob, so adding this field is backwards-compatible
        # (existing rows without it default to FORWARD).
        delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)

        # Where matched videos / link messages are sent. Accepts:
        #   - "me" (default): Saved Messages ? exact v1.1 behaviour.
        #   - "@username": a public channel the account is a member of.
        #   - "-100..." or a bare int-as-string: a numeric Telegram chat id
        #     (private channels without a username).
        # The string is passed verbatim to Telethon's forward_messages / send_message.
        # See ADR 0005 for rationale and access requirements.
        delivery_target: str = Field(
            default="me",
            min_length=1,
            description="Telethon entity to deliver to: 'me', '@username', or a chat id",
        )

        @field_validator("delivery_target", mode="before")
        @classmethod
        def _coerce_delivery_target(cls, v: object) -> str:
            """Accept int chat ids (e.g. -1001234567890) and coerce to str.

            Telethon accepts both str and int as an entity, but storing as str keeps
            the JSON blob uniform and lets '@username' / 'me' / numeric ids share
            one field type.
            """
            if v is None:
                return "me"
            if isinstance(v, int):
                return str(v)
            return str(v)

        @model_validator(mode="after")
        def _check_duration_bounds(self) -> FilterConfig:
            """Ensure min_duration_s <= max_duration_s when both are set."""
            if (
                self.min_duration_s is not None
                and self.max_duration_s is not None
                and self.min_duration_s > self.max_duration_s
```

```
375             ):
376                 raise ValueError(
377                     f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
({self.max_duration_s})"
378                 )
379             return self
380 
381 
382     class VideoRecord(BaseModel):
383         """A seen video message + its metadata. Drives dedup and future /search.
384 
385         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
386         New code should use :class:`ContentItem`; the ``db`` layer maps
387         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
388         """
389 
390         model_config = ConfigDict(frozen=True)
391 
392         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
393         user_id: int
394         channel_id: int = Field(description="Origin Telegram channel id")
395         message_id: int = Field(description="Telegram message id within the channel")
396         document_id: int | None = Field(
397             default=None,
398             description="Telegram document.id ? reused across forwards; Tier-1 content
dedup",
399         )
400 
401         # Metadata extracted from the Telethon message (may be None if absent).
402         duration_s: int | None = None
403         width: int | None = None
404         height: int | None = None
405         size_bytes: int | None = None
406         mime_type: str | None = None
407 
408         matched: bool = False
409         forwarded: bool = False
410         seen_at: datetime = Field(default_factory=_utcnow)
411 
412         @property
413         def dedup_key(self) -> tuple[int, int, int]:
414             """The natural key used for deduplication in the ``videos`` table."""
415             return (self.user_id, self.channel_id, self.message_id)
416 
417         @property
418         def content_fingerprint(self) -> tuple[int, int, int, int] | None:
419             """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
420 
421             Returns None if any field is missing (cannot fingerprint).
422             """
423             if None in (self.size_bytes, self.duration_s, self.height, self.width):
424                 return None
425             return (self.size_bytes, self.duration_s, self.height, self.width)  # type:
ignore[return-value]
426 
427 
428     # Default filter config shared by the bootstrap onboarding step.
429     DEFAULT_FILTER_CONFIG = FilterConfig()
430 
```

7. user (2026-07-06T07:52:34.248Z)

```
1       """Delivery: send matched videos to the user's Saved Messages.
2 
```

```
    Three delivery modes (per-user, configured via FilterConfig.delivery_mode):

    - ``FORWARD``: forward the video itself (re-uploads to Saved Messages).
    - ``LINK``: send a text message with a t.me deep link + metadata summary.
    - ``BOTH``: forward the video AND send the link message.

    All modes wrap their Telethon calls with ``FloodWaitError`` handling (sleep
    ``seconds + jitter``, retry once) and mark the DB row ``forwarded=1`` on success.
    """

    from __future__ import annotations

    import asyncio
    import logging
    import random
    from collections.abc import Awaitable, Callable

    import aiosqlite
    from telethon import TelegramClient
    from telethon.errors import FloodWaitError
    from telethon.tl.types import Message

    from tg_video_filter import db
    from tg_video_filter.models import DeliveryMode

    log = logging.getLogger(__name__)

    # Extra jitter (seconds) added on top of the requested flood wait so we don't
    # retry at the exact boundary and risk re-triggering the limit.
    _FLOOD_JITTER_RANGE = (1.0, 5.0)


    # --- pure link-message formatting ------------------------------------------


    def _format_duration(seconds: int | None) -> str:
        """Human-readable duration for the link summary."""
        if seconds is None:
            return "?"
        if seconds < 60:
            return f"{seconds}s"
        if seconds < 3600:
            return f"{seconds // 60}min"
        h = seconds // 3600
        m = (seconds % 3600) // 60
        return f"{h}h{m:02d}min" if m else f"{h}h"


    def _format_size(size_bytes: int | None) -> str:
        """Human-readable file size for the link summary."""
        if size_bytes is None:
            return "?"
        mib = size_bytes / (1024 * 1024)
        if mib >= 1024:
            return f"{mib / 1024:.1f}GB"
        return f"{mib:.0f}MB"


    def _format_height(height: int | None) -> str:
        """Resolution label: 1080 -> 1080p, None -> ?."""
        if height is None:
            return "?"
        return f"{height}p"


```

```
    def build_link_text(
        *,
        message_link: str | None,
        duration_s: int | None,
        height: int | None,
        size_bytes: int | None,
        channel_title: str | None,
    ) -> str:
        """Build the text message sent in LINK/BOTH mode.

        Format::

            ? 1080p · 5min · 480MB
            #ChannelName
            https://t.me/c/1234567890/42

        ``message_link`` may be None if the link can't be constructed (e.g. the
        channel is inaccessible); in that case the link line is omitted.
        """
        lines = [
            f"? {_format_height(height)} · {_format_duration(duration_s)} ·
{_format_size(size_bytes)}"
        ]
        if channel_title:
            # Sanitize: replace spaces in the title for a hashtag-like display.
            lines.append(f"#{channel_title.replace(' ', '_')}")
        if message_link:
            lines.append(message_link)
        return "\n".join(lines)


    # --- Telethon interaction (retry/flood logic) --------------------------------


    async def _send_with_retry(
        client: TelegramClient,
        send_callable: Callable[[], Awaitable[None]],
        video_id: int,
    ) -> bool:
        """Execute a send/forward callable with FloodWait handling. Returns success."""
        try:
            await send_callable()
        except FloodWaitError as e:
            wait = e.seconds + random.uniform(*_FLOOD_JITTER_RANGE)
            log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id,
e.seconds, wait)
            await asyncio.sleep(wait)
            try:
                await send_callable()
            except Exception:
                log.exception("delivery_retry_failed video=%s", video_id)
                return False
        except Exception:
            log.exception("delivery_failed video=%s", video_id)
            return False
        return True


    async def forward_to_target(
        client: TelegramClient,
        message: Message,
        conn: aiosqlite.Connection,
        video_id: int,
        target: str = "me",
        *,
```

```
        destination_id: int | None = None,
    ) -> bool:
        """Forward ``message`` to ``target``, recording delivery state.

        ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"``,
        or a numeric chat id string. See ADR 0005.

        If ``destination_id`` is provided, a per-destination ``DeliveryAttempt``
        is written (ADR 0011 Track C). Otherwise falls back to the legacy no-op
        ``mark_forwarded``.
        """
        ok = await _send_with_retry(client, lambda: client.forward_messages(target,
message), video_id)
        if ok:
            if destination_id is not None:
                await db.record_delivery(
                    conn, content_item_id=video_id, destination_id=destination_id,
success=True
                )
            else:
                await db.mark_forwarded(conn, video_id)
            log.info("video_forwarded video=%s target=%s", video_id, target)
        elif destination_id is not None:
            await db.record_delivery(
                conn, content_item_id=video_id, destination_id=destination_id, success=False
            )
        return ok


    async def send_link_message(
        client: TelegramClient,
        text: str,
        conn: aiosqlite.Connection,
        video_id: int,
        target: str = "me",
        *,
        destination_id: int | None = None,
    ) -> bool:
        """Send a link/summary text message to ``target``, recording delivery state.

        ``target`` is a Telethon entity; see :func:`forward_to_target`.
        """
        ok = await _send_with_retry(client, lambda: client.send_message(target, text),
video_id)
        if ok:
            if destination_id is not None:
                await db.record_delivery(
                    conn, content_item_id=video_id, destination_id=destination_id,
success=True
                )
            else:
                await db.mark_forwarded(conn, video_id)
            log.info("link_sent video=%s target=%s", video_id, target)
        elif destination_id is not None:
            await db.record_delivery(
                conn, content_item_id=video_id, destination_id=destination_id, success=False
            )
        return ok


    async def deliver(
        client: TelegramClient,
        message: Message,
        conn: aiosqlite.Connection,
        video_id: int,
```

```
        mode: DeliveryMode,
        *,
        link_text: str | None = None,
        target: str = "me",
        destination_id: int | None = None,
    ) -> bool:
        """Dispatch delivery based on ``mode`` to ``target``.

        ``link_text`` is required for LINK/BOTH modes (built by the caller via
        :func:`build_link_text`). If it's None in LINK/BOTH mode, falls back to
        FORWARD so we never silently drop a matched video.

        ``target`` is a Telethon entity (``"me"``, ``"@username"``, chat id).
        See ADR 0005.

        ``destination_id`` is the FK to ``destinations.id`` (ADR 0011 Track C).
        When provided, a per-destination ``DeliveryAttempt`` is written.
        """
        if mode == DeliveryMode.LINK and link_text is not None:
            return await send_link_message(
                client, link_text, conn, video_id, target, destination_id=destination_id
            )
        if mode == DeliveryMode.BOTH and link_text is not None:
            fwd_ok = await forward_to_target(
                client, message, conn, video_id, target, destination_id=destination_id
            )
            link_ok = await send_link_message(
                client, link_text, conn, video_id, target, destination_id=destination_id
            )
            if not link_ok:
                log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
            if not fwd_ok:
                log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
            return fwd_ok or link_ok
        # FORWARD, or fallback when link_text is missing.
        if mode != DeliveryMode.FORWARD:
            log.warning(
                "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
                video_id,
                mode.value,
                link_text is None,
            )
        return await forward_to_target(
            client, message, conn, video_id, target, destination_id=destination_id
        )


    __all__ = [
        "build_link_text",
        "deliver",
        "forward_to_target",
        "send_link_message",
    ]

```

8. assistant (2026-07-06T07:52:36.906Z)

Now let me look at the schema definitions for these tables and the repo functions (upsert_destination, record_delivery, etc.).

9. user (2026-07-06T07:52:38.741Z)

```
143:CREATE TABLE IF NOT EXISTS destinations (
155:CREATE TABLE IF NOT EXISTS routing_rules (
167:CREATE TABLE IF NOT EXISTS delivery_attempts (
```

```
1275:async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) -> Destination:
1307:async def get_destination(
1320:async def upsert_delivery_attempt(
1358:async def record_delivery(
1399:async def get_delivery_attempt(
1754:    "upsert_destination",
1755:    "get_destination",
1756:    "upsert_delivery_attempt",
1757:    "record_delivery",
1758:    "get_delivery_attempt",
```

10. user (2026-07-06T07:52:41.647Z)

```
130         account_id  INTEGER NOT NULL REFERENCES accounts(id),
131         name        TEXT NOT NULL,
132         config_json TEXT NOT NULL,
133         is_default  INTEGER NOT NULL DEFAULT 0,
134         created_at  TEXT NOT NULL,
135         updated_at  TEXT NOT NULL,
136         UNIQUE (account_id, name)
137     );
138
139     -- ADR 0011: exactly one default policy per account (structural invariant).
140     CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
141     ON policies(account_id) WHERE is_default = 1;
142
143     CREATE TABLE IF NOT EXISTS destinations (
144         id          INTEGER PRIMARY KEY AUTOINCREMENT,
145         account_id  INTEGER NOT NULL REFERENCES accounts(id),
146         provider    TEXT NOT NULL DEFAULT 'telegram',
147         target      TEXT NOT NULL,
148         kind        TEXT NOT NULL DEFAULT 'saved',
149         title       TEXT,
150         is_output   INTEGER NOT NULL DEFAULT 0,
151         created_at  TEXT NOT NULL,
152         UNIQUE (account_id, provider, target)
153     );
154
155     CREATE TABLE IF NOT EXISTS routing_rules (
156         id             INTEGER PRIMARY KEY AUTOINCREMENT,
157         account_id     INTEGER NOT NULL REFERENCES accounts(id),
158         source_id      INTEGER NOT NULL REFERENCES sources(id),
159         policy_id      INTEGER REFERENCES policies(id),
160         destination_id INTEGER NOT NULL REFERENCES destinations(id),
161         enabled        INTEGER NOT NULL DEFAULT 1,
162         priority       INTEGER NOT NULL DEFAULT 0,
163         created_at     TEXT NOT NULL,
164         UNIQUE (account_id, source_id, destination_id)
165     );
166
167     CREATE TABLE IF NOT EXISTS delivery_attempts (
168         id              INTEGER PRIMARY KEY AUTOINCREMENT,
169         content_item_id INTEGER NOT NULL REFERENCES content_items(id),
170         destination_id  INTEGER NOT NULL REFERENCES destinations(id),
171         route_id        INTEGER REFERENCES routing_rules(id),
172         status          TEXT NOT NULL DEFAULT 'pending',
173         attempt_count   INTEGER NOT NULL DEFAULT 0,
174         last_error      TEXT,
175         created_at      TEXT NOT NULL,
176         delivered_at    TEXT,
177         next_retry_at   TEXT,
178         UNIQUE (content_item_id, destination_id)
179     );
180     """
181
```

```
182     # Note: the ``idx_videos_user_document`` partial unique index is created by
183     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
184     # which is present on fresh tables but only added to legacy v1 DBs by the
185     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
186     # DBs that don't yet have the column.
187
188
189     async def init_db(db_path: str) -> None:
```

11. user (2026-07-06T07:52:43.617Z)

```
1275    async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) ->
Destination:
1276        """Insert or update a destination row by (account, provider, target)."""
1277        await conn.execute(
1278            """INSERT INTO destinations
1279                 (account_id, provider, target, kind, title, is_output, created_at)
1280               VALUES (?, ?, ?, ?, ?, ?, ?)
1281               ON CONFLICT(account_id, provider, target) DO UPDATE SET
1282                 kind=excluded.kind,
1283                 title=excluded.title,
1284                 is_output=excluded.is_output""",
1285            (
1286                dest.account_id,
1287                dest.provider,
1288                dest.target,
1289                dest.kind,
1290                dest.title,
1291                int(dest.is_output),
1292                dest.created_at.isoformat(),
1293            ),
1294        )
1295        await conn.commit()
1296        conn.row_factory = aiosqlite.Row
1297        async with conn.execute(
1298            """SELECT * FROM destinations
1299               WHERE account_id = ? AND provider = ? AND target = ?""",
1300            (dest.account_id, dest.provider, dest.target),
1301        ) as cur:
1302            row = await cur.fetchone()
1303        assert row is not None
1304        return _row_to_destination(row)
1305
1306
1307    async def get_destination(
1308        conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1309    ) -> Destination | None:
1310        """Resolve a destination by its (account, provider, target) natural key."""
1311        conn.row_factory = aiosqlite.Row
1312        async with conn.execute(
1313            "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target =
?",
1314            (account_id, provider, target),
1315        ) as cur:
1316            row = await cur.fetchone()
1317        return _row_to_destination(row) if row else None
1318
1319
1320    async def upsert_delivery_attempt(
1321        conn: aiosqlite.Connection, attempt: DeliveryAttempt
1322    ) -> DeliveryAttempt:
1323        """Insert or update a delivery_attempt; idempotent on (content_item,
destination)."""
1324        await conn.execute(
1325            """INSERT INTO delivery_attempts
```

```
                (content_item_id, destination_id, route_id, status, attempt_count,
                 last_error, created_at, delivered_at, next_retry_at)
              VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
              ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
                status=excluded.status,
                attempt_count=excluded.attempt_count,
                last_error=excluded.last_error,
                delivered_at=excluded.delivered_at,
                next_retry_at=excluded.next_retry_at""",
            (
                attempt.content_item_id,
                attempt.destination_id,
                attempt.route_id,
                attempt.status.value,
                attempt.attempt_count,
                attempt.last_error,
                attempt.created_at.isoformat(),
                attempt.delivered_at.isoformat() if attempt.delivered_at else None,
                attempt.next_retry_at.isoformat() if attempt.next_retry_at else None,
            ),
        )
        await conn.commit()
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
?",
            (attempt.content_item_id, attempt.destination_id),
        ) as cur:
            row = await cur.fetchone()
        assert row is not None
        return _row_to_delivery_attempt(row)


    async def record_delivery(
        conn: aiosqlite.Connection,
        *,
        content_item_id: int,
        destination_id: int,
        route_id: int | None = None,
        success: bool,
        error: str | None = None,
    ) -> None:
        """Record a delivery outcome for one (content_item, destination) pair.

        This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
        sets status=delivered; on failure, increments attempt_count and records the
        error. Idempotent: repeated calls update the same row.
        """
        status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
        await conn.execute(
            """INSERT INTO delivery_attempts
                 (content_item_id, destination_id, route_id, status, attempt_count,
                  last_error, created_at, delivered_at)
               VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
                       CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
               ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
                 status=excluded.status,
                 attempt_count=delivery_attempts.attempt_count + 1,
                 last_error=excluded.last_error,
                 delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
                                   ELSE delivery_attempts.delivered_at END""",
            (
                content_item_id,
                destination_id,
                route_id,
```

```
1390                status.value,
1391                error,
1392                success,
1393                success,
1394            ),
1395        )
1396        await conn.commit()
1397
1398
1399    async def get_delivery_attempt(
1400        conn: aiosqlite.Connection, content_item_id: int, destination_id: int
1401    ) -> DeliveryAttempt | None:
1402        """Read a delivery attempt row by its natural key (idempotency boundary)."""
1403        conn.row_factory = aiosqlite.Row
1404        async with conn.execute(
1405            "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
?",
1406            (content_item_id, destination_id),
1407        ) as cur:
1408            row = await cur.fetchone()
1409        return _row_to_delivery_attempt(row) if row else None
1410
1411
1412    # --- legacy Channel shim (backwards-compatible view over sources) ------------
1413
1414
1415    async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1416        """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1417
1418        ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1419        reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1420        New code should call :func:`upsert_source` directly.
1421        """
1422        src = await upsert_source(
1423            conn,
1424            Source(
1425                account_id=channel.user_id,
1426                provider="telegram",
1427                provider_source_id=str(channel.channel_id),
1428                title=channel.title,
1429                kind="channel",
1430                watched=True,  # back-compat: pre-ADR-0011 channels were all watched
1431                seen_first_at=channel.seen_first_at,
1432            ),
1433        )
1434        return _source_to_channel(src)
```

12. assistant (2026-07-06T07:52:48.518Z)

Now let me look at the migration ladder mechanism and the videos_deprecated / content_items schema to check the delivery_attempts backfill JOIN correctness. Let me check the `_migration` decorator and how user_version gating works.

13. user (2026-07-06T07:52:50.025Z)

```
193:    then runs any pending migrations via the ``user_version`` ladder.
199:async def _apply_schema(conn: aiosqlite.Connection) -> None:
222:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
228:#   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
229:#   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
233:_MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
236:def _migration(
242:    """Register a migration that brings the DB up to ``user_version == version``."""
247:        _MIGRATIONS.append((version, fn))
316:    """Run every registered migration whose version > current ``user_version``."""
```

```
317:    async with conn.execute("PRAGMA user_version") as cur:
321:    for version, fn in _MIGRATIONS:
325:        await conn.execute(f"PRAGMA user_version = {version}")
336:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
353:async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
505:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
668:async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
```

14. user (2026-07-06T07:52:57.773Z)

```
305                 )
306             return f"tg:msg:{source_id}:{provider_message_id}"
307         # Future-proof default for providers without a dedicated branch.
308         if source_id is None or provider_message_id is None:
309             raise ValueError(
310                 f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
311             )
312         return f"{p}:msg:{source_id}:{provider_message_id}"
313
314
315     async def _apply_migrations(conn: aiosqlite.Connection) -> None:
316         """Run every registered migration whose version > current ``user_version``."""
317         async with conn.execute("PRAGMA user_version") as cur:
318             row = await cur.fetchone()
319         current = int(row[0]) if row else 0
320
321         for version, fn in _MIGRATIONS:
322             if version <= current:
323                 continue
324             await fn(conn)
325             await conn.execute(f"PRAGMA user_version = {version}")
326
327
328     async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
329         """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
330         async with conn.execute(f"PRAGMA table_info({table})") as cur:
331             rows = await cur.fetchall()
332         return any(row[1] == column for row in rows)
333
334
335     @_migration(1)
336     async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
337         """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
338
339         Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
```

15. user (2026-07-06T07:52:58.305Z)

```
505     async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
506         """Land the provider-neutral domain tables (ADR 0011, Track B).
507
508         Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
509         ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
510         DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
511         them), then backfills the three populated ones from the legacy
512         ``channels`` / ``videos`` / ``filter_config`` tables and renames the
513         legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
514         pattern). Idempotent: a no-op once the legacy tables have been renamed.
515         """
516         # Idempotency gate: if the legacy videos table is gone, this migration has
517         # already run (or the DB is fresh and _SCHEMA's legacy tables were already
518         # dropped by _apply_schema post-003). Nothing to backfill.
519         if not await _table_exists(conn, "videos"):
520             return
```

```
521
522         # 1a. Backfill sources from channels first (known title + watched=1). Order
523         #    matters: channels-first so title/watched data wins; the videos-based
524         #    backfill below fills gaps for orphan channels (review finding #1).
525         await conn.execute(
526             """
527             INSERT OR IGNORE INTO sources
528                 (account_id, provider, provider_source_id, title, kind, watched,
529                  seen_first_at, last_seen_at)
530             SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
531                    1, seen_first_at, seen_first_at
532             FROM channels
533             """
534         )
535
536         # 1b. Backfill sources from videos for channels that have no channels row
537         #     (e.g. backfill-only channels that were never seen via live traffic).
538         #     INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
539         #     ? the channels-based row above survives with its title + watched=1.
540         await conn.execute(
541             """
542             INSERT OR IGNORE INTO sources
543                 (account_id, provider, provider_source_id, kind, watched, seen_first_at)
544             SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
545                    'channel', 0, MIN(v.seen_at)
546             FROM videos v
547             GROUP BY v.user_id, v.channel_id
548             """
549         )
550
551         # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
552         #    Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
553         #    (no silent drops, even on fingerprint collisions).
554         #    Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
group).
555         #    Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
group).
556         #    The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
557         await conn.execute(
558             """
559             INSERT INTO content_items
560                 (account_id, source_id, provider, provider_message_id, content_type,
561                  document_id, duration_s, width, height, size_bytes, mime_type,
562                  dedup_key, matched, seen_at)
563             SELECT
564                 v.user_id,
565                 s.id,
566                 'telegram',
567                 CAST(v.message_id AS TEXT),
568                 'video',
569                 CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
570                 v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
571                 'tg:msg:' || s.id || ':' || v.message_id,  -- Tier-0 (always unique)
572                 v.matched,
573                 v.seen_at
574             FROM videos v
575             JOIN sources s
576               ON s.account_id = v.user_id
577              AND s.provider = 'telegram'
578              AND s.provider_source_id = CAST(v.channel_id AS TEXT)
579             """
580         )
581
582         # Phase B: promote one row per (account, document_id) to tg:doc:.
583         await conn.execute(
```

```
584             """
585             UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
586             WHERE document_id IS NOT NULL
587               AND dedup_key LIKE 'tg:msg:%'
588               AND id IN (
589                 SELECT MIN(id) FROM content_items
590                 WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
591                 GROUP BY account_id, document_id
592               )
593             """
594         )
595 
596         # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
597         await conn.execute(
598             """
599             UPDATE content_items SET dedup_key =
600                 'tg:fp:' || size_bytes || ':' || duration_s || ':'
601                 || height || ':' || width
602             WHERE document_id IS NULL
603               AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
604               AND height IS NOT NULL AND width IS NOT NULL
```

16. assistant (2026-07-06T07:53:02.225Z)

Now I understand the migration structure. Key observations for the delivery_attempts backfill: content_items provider_message_id is only unique within (account, source), but the JOIN on line 736-739 joins on `ci.provider_message_id = CAST(vd.message_id AS TEXT)` AND `ci.account_id = vd.user_id` ? but NOT on source. Let me check this and the transaction/atomicity handling. Let me look at how `_apply_migrations` is wrapped (transaction) and check `init_db`.

17. user (2026-07-06T07:53:03.557Z)

```
189     async def init_db(db_path: str) -> None:
190         """Open the database at ``db_path`` and create/migrate the schema.
191 
192         Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
193         then runs any pending migrations via the ``user_version`` ladder.
194         """
195         async with aiosqlite.connect(db_path) as conn:
196             await _apply_schema(conn)
197 
198 
199     async def _apply_schema(conn: aiosqlite.Connection) -> None:
200         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
201         # and a busy timeout so concurrent access waits instead of failing immediately.
202         await conn.execute("PRAGMA journal_mode = WAL")
203         await conn.execute("PRAGMA foreign_keys = ON")
204         await conn.execute("PRAGMA busy_timeout = 5000")
205         await conn.executescript(_SCHEMA)
206         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
207         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
208         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
209         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
210         # any *empty* recreations of those legacy tables from a subsequent
211         # ``init_db`` are dropped here so they don't shadow the renamed copies.
212         if await _table_exists(conn, "channels_deprecated"):
213             await conn.execute("DROP TABLE IF EXISTS channels")
214             await conn.execute("DROP TABLE IF EXISTS videos")
215             await conn.execute("DROP TABLE IF EXISTS filter_config")
216         await _apply_migrations(conn)
217         await conn.commit()
218 
219 
220     # --- migrations ----------------------------------------------------------
221     #
```

```
222     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
223     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
224     # the source of truth for *fresh* databases; the migrations below reconcile
225     # *existing* databases that were created by an older schema version.
226     #
227     # To add a new migration:
228     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
229     #   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
230     #   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
231     #      ``init_db``, and asserts the new structure.
232
233     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
```

18. assistant (2026-07-06T07:53:09.149Z)

Critical atomicity observation: `_apply_migrations` runs all migrations and only commits once at the end of `_apply_schema` (line 217). But migration 004's own repo helpers commit... no, migration 004 uses raw `conn.execute` only (no commit inside). However, migration 003 renames tables. Let me check whether there's an implicit commit issue. Actually the bigger concern from the prompt: the idempotency gate uses `COUNT(destinations)>0` rather than user_version. But `_apply_migrations` sets user_version AFTER each migration succeeds (line 325). So if 004 crashes mid-way, user_version stays at 3, and 004 re-runs. Let me analyze the partial-run scenario carefully with a repro.

Let me now build a repro to test the key scenarios. First let me check the test files to understand seeding helpers.

19. user (2026-07-06T07:53:10.669Z)

```
1       """Tests for the SQLite data layer (db.py) using an in-memory database."""
2
3       from __future__ import annotations
4
5       import sqlite3
6
7       import aiosqlite
8       import pytest
9
10      from tg_video_filter import db
11      from tg_video_filter.models import (
12          DEFAULT_FILTER_CONFIG,
13          Account,
14          Channel,
15          ContentItem,
16          DeliveryStatus,
17          Destination,
18          FilterConfig,
19          Policy,
20          Source,
21          TelegramSession,
22          User,
23          VideoRecord,
24      )
25
26      USER_ID = 111
27      CHANNEL_ID = -1001234567890
28
29
30      @pytest.fixture
31      async def conn() -> aiosqlite.Connection:
32          """An in-memory SQLite connection with the schema applied."""
33          c = await aiosqlite.connect(":memory:")
34          await db._apply_schema(c)
35          yield c
36          await c.close()
```

```
37
38
39      async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
40          return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
41
42
43      # --- schema / init ---------------------------------------------------------
44
45
46      async def test_init_db_creates_tables(tmp_path: object) -> None:
47          db_path = str(tmp_path) + "/test.db"  # type: ignore[operator]
48          await db.init_db(db_path)
49          # Re-open and confirm all expected tables exist. Post-ADR-0011 the
50          # provider-neutral tables are sources/content_items/policies (plus the
51          # empty Track-C tables destinations/routing_rules/delivery_attempts);
52          # the legacy channels/videos/filter_config are absent on fresh DBs
53          # (they only exist as *_deprecated on migrated legacy DBs).
54          async with (
55              aiosqlite.connect(db_path) as c,
56              c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
57          ):
58              names = [r[0] for r in await cur.fetchall()]
59          assert set(names) >= {
60              "accounts",
```

20. user (2026-07-06T07:53:12.255Z)

```
39:async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
52:    # the legacy channels/videos/filter_config are absent on fresh DBs
53:    # (they only exist as *_deprecated on migrated legacy DBs).
109:async def _seed_account(conn: aiosqlite.Connection, aid: int = USER_ID) -> Account:
186:    """A bot-only account is a real Account but not a legacy 'enabled user'."""
194:    legacy = await db.get_user(conn, USER_ID)
195:    assert legacy is not None
196:    assert legacy.enabled is False
627:async def test_fresh_db_user_version_is_latest(conn: aiosqlite.Connection) -> None:
628:    """init_db/_apply_schema must bump user_version to the highest migration."""
630:    async with conn.execute("PRAGMA user_version") as cur:
636:async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
637:    """A v1 DB (no document_id, user_version=0) must be migrated on open."""
638:    db_path = str(tmp_path) + "/legacy.db"  # type: ignore[operator]
640:    # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
641:    # no document_id column, user_version = 0. Using the real FK constraints
645:    legacy_schema = """
646:    CREATE TABLE users (
650:    CREATE TABLE channels (
656:    CREATE TABLE videos (
665:    CREATE TABLE filter_config (
670:        await conn.executescript(legacy_schema)
686:        # user_version is 0 on a freshly created DB.
693:        # Column added (migration 001) ? on the legacy videos table, now renamed.
704:        # telegram_sessions row derived from the legacy user row.
711:        # Domain schema (migration 003): legacy tables renamed to *_deprecated,
723:        # FK contract (PR #33 review): the rebuilt legacy tables (now
775:        # user_version bumped to the highest registered migration.
776:        async with conn.execute("PRAGMA user_version") as cur:
787:    db_path = str(tmp_path) + "/legacy_fk.db"  # type: ignore[operator]
788:    legacy_schema = """
789:    CREATE TABLE users (
793:    CREATE TABLE channels (
802:        await conn.executescript(legacy_schema)
842:        async with conn.execute("PRAGMA user_version") as cur:
1118:# --- migration 003: legacy data backfill (ADR 0011) -------------------------
1124:    to the same key. Seeds rows hitting each branch on a legacy DB.
```

```
1126:    db_path = str(tmp_path) + "/legacy_dedup.db"  # type: ignore[operator]
1127:    legacy_schema = """
1128:    CREATE TABLE users (

[Showing results with pagination = limit: 40]
```

21. assistant (2026-07-06T07:53:16.492Z)

Let me read the legacy schema seeding in the migration 003/004 tests to build a proper repro.

22. user (2026-07-06T07:53:17.320Z)

```
636     async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
637         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
638         db_path = str(tmp_path) + "/legacy.db"  # type: ignore[operator]
639
640         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
641         # no document_id column, user_version = 0. Using the real FK constraints
642         # is what lets this test catch the PR #33 regression where SQLite rewrites
643         # FK references to follow a rename (users ? users_deprecated) instead of
644         # retargeting them to accounts.
645         legacy_schema = """
646         CREATE TABLE users (
647             id INTEGER PRIMARY KEY, phone TEXT,
648             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
649         );
650         CREATE TABLE channels (
651             id INTEGER PRIMARY KEY AUTOINCREMENT,
652             user_id INTEGER NOT NULL REFERENCES users(id),
653             channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
654             UNIQUE (user_id, channel_id)
655         );
656         CREATE TABLE videos (
657             id INTEGER PRIMARY KEY AUTOINCREMENT,
658             user_id INTEGER NOT NULL REFERENCES users(id),
659             channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
660             duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
661             mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
662             forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
663             UNIQUE (user_id, channel_id, message_id)
664         );
665         CREATE TABLE filter_config (
666             user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
667         );
668         """
669         async with aiosqlite.connect(db_path) as conn:
670             await conn.executescript(legacy_schema)
671             # Seed rows so we can confirm data survives the migration.
672             await conn.execute(
673                 "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z')"
674             )
675             await conn.execute(
676                 "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
677                 "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
678             )
679             await conn.execute(
680                 "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
height, "
681                 "size_bytes, mime_type, matched, forwarded, seen_at) "
682                 "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')"
683             )
684             await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES
(111, '{}')")
```

```
685             await conn.commit()
686             # user_version is 0 on a freshly created DB.
687
688         # Now open via init_db ? migrations 001, 002, and 003 must run.
689         await db.init_db(db_path)
690
691         async with aiosqlite.connect(db_path) as conn:
692             await conn.execute("PRAGMA foreign_keys=ON")
693             # Column added (migration 001) ? on the legacy videos table, now renamed.
694             assert await db._column_exists(conn, "videos_deprecated", "document_id")
695             # Identity split (migration 002): users renamed, accounts present.
696             assert await db._table_exists(conn, "accounts")
697             assert await db._table_exists(conn, "telegram_sessions")
698             assert await db._table_exists(conn, "users_deprecated")
699             assert not await db._table_exists(conn, "users")
700             # accounts.id copied from users.id (integer-preserving).
701             async with conn.execute("SELECT id, telegram_id, provider FROM accounts") as
cur:
702                 rows = await cur.fetchall()
703             assert rows == [(111, 111, "telegram")]
704             # telegram_sessions row derived from the legacy user row.
705             async with conn.execute(
706                 "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
707             ) as cur:
708                 srows = await cur.fetchall()
709             assert srows == [(111, "+1", "user_111.session", 1)]
710
711             # Domain schema (migration 003): legacy tables renamed to *_deprecated,
712             # provider-neutral tables backfilled from them.
713             assert await db._table_exists(conn, "sources")
714             assert await db._table_exists(conn, "content_items")
715             assert await db._table_exists(conn, "policies")
716             assert await db._table_exists(conn, "channels_deprecated")
717             assert await db._table_exists(conn, "videos_deprecated")
718             assert await db._table_exists(conn, "filter_config_deprecated")
719             assert not await db._table_exists(conn, "channels")
720             assert not await db._table_exists(conn, "videos")
721             assert not await db._table_exists(conn, "filter_config")
722
723             # FK contract (PR #33 review): the rebuilt legacy tables (now
724             # *_deprecated) reference accounts(id), NOT users_deprecated(id).
725             for tbl in ("channels_deprecated", "videos_deprecated",
"filter_config_deprecated"):
726                 async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
727                     fk_rows = await cur.fetchall()
728                 referenced_tables = {r[2] for r in fk_rows}
729                 assert "accounts" in referenced_tables, f"{tbl} FK does not reference
accounts"
730                 assert (
731                     "users_deprecated" not in referenced_tables
732                 ), f"{tbl} FK still references users_deprecated (rename-rewrite bug)"
733
734             # Legacy data preserved in the deprecated tables.
735             async with conn.execute(
```

23. assistant (2026-07-06T07:53:25.238Z)

Now I have enough to build a repro. Let me write a script to test the key scenarios: (1) multiple sources/multiple destinations cartesian, (2) numeric chat id kind classification, (3) no-filter-config account, (4) delivery_attempts JOIN correctness across sources, (5) partial-run idempotency.

Let me focus on the most likely real bug: the delivery_attempts backfill JOIN. content_items has UNIQUE(account_id, source_id, provider_message_id) ? message_id is per-channel. But the JOIN at line 736-739 matches `ci.account_id = vd.user_id AND ci.provider_message_id = CAST(vd.message_id

AS TEXT)` WITHOUT constraining `ci.source_id = sc.id`. It separately joins `sc ON sc.id =
ci.source_id`. Let me trace: if the same message_id exists in two different channels for one
account, the join would produce cross-products. Let me build the repro.

24. assistant (2026-07-06T07:53:42.975Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db

LEGACY = """
CREATE TABLE users (
    id INTEGER PRIMARY KEY, phone TEXT,
    enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
);
CREATE TABLE channels (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
    UNIQUE (user_id, channel_id)
);
CREATE TABLE videos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
    duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
    mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
    forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
    UNIQUE (user_id, channel_id, message_id)
);
CREATE TABLE filter_config (
    user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
);
"""

async def seed(path, fc_json, videos, channels):
    async with aiosqlite.connect(path) as conn:
        await conn.executescript(LEGACY)
        await conn.execute("INSERT INTO users (id,phone,enabled,created_at) VALUES
(111,'+1',1,'2024-01-01T00:00:00Z')")
        for ch, title in channels:
            await conn.execute("INSERT INTO channels (user_id,channel_id,title,seen_first_at)
VALUES (111,?,?,'2024-01-01T00:00:00Z')",(ch,title))
        for (ch,mid,fwd) in videos:
            await conn.execute("INSERT INTO videos (user_id,channel_id,message_id,duration_s,wid
th,height,size_bytes,mime_type,matched,forwarded,seen_at) VALUES
(111,?,?,60,1920,1080,1000000,'video/mp4',1,?,'2024-01-01T00:00:00Z')",(ch,mid,fwd))
        if fc_json is not None:
            await conn.execute("INSERT INTO filter_config (user_id,config_json) VALUES
(111,?)",(fc_json,))
        await conn.commit()

async def dump(path, label):
    async with aiosqlite.connect(path) as conn:
        conn.row_factory = aiosqlite.Row
        print(f"\n=== {label} ===")
        for t in ("destinations","routing_rules","delivery_attempts"):
            async with conn.execute(f"SELECT * FROM {t}") as cur:
                rows = await cur.fetchall()
            print(f"{t}: {len(rows)} rows")
            for r in rows:
                print("   ", dict(r))

async def scenario(name, path, fc_json, videos, channels):
    import os
```

```
    if os.path.exists(path): os.remove(path)
    await seed(path, fc_json, videos, channels)
    await db.init_db(path)
    await dump(path, name)

async def main():
    base="C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/"

    # Scenario A: same message_id across two channels, both forwarded.
    # Tests whether delivery_attempts JOIN produces cross-source duplicates.
    await scenario("A: same msg_id in 2 channels", base+"a.db", '{"delivery_target":"me"}',
        videos=[(-1001,5,1),(-1002,5,1)], channels=[(-1001,"chA"),(-1002,"chB")])

    # Scenario B: numeric chat id delivery_target -> kind classification
    await scenario("B: numeric chat id target", base+"b.db",
'{"delivery_target":"-1009999999999"}',
        videos=[(-1001,5,1)], channels=[(-1001,"chA")])

    # Scenario C: no filter_config row at all
    await scenario("C: no filter_config", base+"c.db", None,
        videos=[(-1001,5,1)], channels=[(-1001,"chA")])

    # Scenario D: @channel target
    await scenario("D: @channel target", base+"d.db", '{"delivery_target":"@mychan"}',
        videos=[(-1001,5,1)], channels=[(-1001,"chA")])

asyncio.run(main())
```

25. user (2026-07-06T07:53:42.976Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

26. assistant (2026-07-06T07:53:45.946Z)

The file doesn't exist yet; this is a spurious guard. Let me use a different filename.

27. user (2026-07-06T07:54:02.293Z)

```
=== A: same msg_id in 2 channels, both fwd ===
destinations: 1 rows
    {'id': 1, 'account_id': 111, 'provider': 'telegram', 'target': 'me', 'kind': 'saved',
'title': None, 'is_output': 0, 'created_at': '2026-07-06T07:54:02Z'}
routing_rules: 2 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'policy_id': 1, 'destination_id': 1, 'enabled':
1, 'priority': 0, 'created_at': '2026-07-06T07:54:02Z'}
    {'id': 2, 'account_id': 111, 'source_id': 2, 'policy_id': 1, 'destination_id': 1, 'enabled':
1, 'priority': 0, 'created_at': '2026-07-06T07:54:02Z'}
delivery_attempts: 2 rows
    {'id': 1, 'content_item_id': 1, 'destination_id': 1, 'route_id': 1, 'status': 'delivered',
'attempt_count': 1, 'last_error': None, 'created_at': '2024-01-01T00:00:00Z', 'delivered_at':
'2024-01-01T00:00:00Z', 'next_retry_at': None}
    {'id': 2, 'content_item_id': 2, 'destination_id': 1, 'route_id': 2, 'status': 'delivered',
'attempt_count': 1, 'last_error': None, 'created_at': '2024-01-01T00:00:00Z', 'delivered_at':
'2024-01-01T00:00:00Z', 'next_retry_at': None}
content_items: 2 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'provider': 'telegram', 'provider_message_id':
'5', 'content_type': 'video', 'text': None, 'link_url': None, 'media_ref': None, 'document_id':
None, 'duration_s': 60, 'width': 1920, 'height': 1080, 'size_bytes': 1000000, 'mime_type':
'video/mp4', 'dedup_key': 'tg:fp:1000000:60:1080:1920', 'matched': 1, 'seen_at':
'2024-01-01T00:00:00Z'}
    {'id': 2, 'account_id': 111, 'source_id': 2, 'provider': 'telegram', 'provider_message_id':
'5', 'content_type': 'video', 'text': None, 'link_url': None, 'media_ref': None, 'document_id':
None, 'duration_s': 60, 'width': 1920, 'height': 1080, 'size_bytes': 1000000, 'mime_type':
```

```
'video/mp4', 'dedup_key': 'tg:msg:2:5', 'matched': 1, 'seen_at': '2024-01-01T00:00:00Z'}

=== B: numeric chat id target ===
destinations: 1 rows
    {'id': 1, 'account_id': 111, 'provider': 'telegram', 'target': '-1009999999999', 'kind':
'channel', 'title': None, 'is_output': 0, 'created_at': '2026-07-06T07:54:02Z'}
routing_rules: 1 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'policy_id': 1, 'destination_id': 1, 'enabled':
1, 'priority': 0, 'created_at': '2026-07-06T07:54:02Z'}
delivery_attempts: 1 rows
    {'id': 1, 'content_item_id': 1, 'destination_id': 1, 'route_id': 1, 'status': 'delivered',
'attempt_count': 1, 'last_error': None, 'created_at': '2024-01-01T00:00:00Z', 'delivered_at':
'2024-01-01T00:00:00Z', 'next_retry_at': None}
content_items: 1 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'provider': 'telegram', 'provider_message_id':
'5', 'content_type': 'video', 'text': None, 'link_url': None, 'media_ref': None, 'document_id':
None, 'duration_s': 60, 'width': 1920, 'height': 1080, 'size_bytes': 1000000, 'mime_type':
'video/mp4', 'dedup_key': 'tg:fp:1000000:60:1080:1920', 'matched': 1, 'seen_at':
'2024-01-01T00:00:00Z'}

=== C: no filter_config ===
destinations: 0 rows
routing_rules: 0 rows
delivery_attempts: 0 rows
content_items: 1 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'provider': 'telegram', 'provider_message_id':
'5', 'content_type': 'video', 'text': None, 'link_url': None, 'media_ref': None, 'document_id':
None, 'duration_s': 60, 'width': 1920, 'height': 1080, 'size_bytes': 1000000, 'mime_type':
'video/mp4', 'dedup_key': 'tg:fp:1000000:60:1080:1920', 'matched': 1, 'seen_at':
'2024-01-01T00:00:00Z'}

=== D: @channel target ===
destinations: 1 rows
    {'id': 1, 'account_id': 111, 'provider': 'telegram', 'target': '@mychan', 'kind': 'channel',
'title': None, 'is_output': 0, 'created_at': '2026-07-06T07:54:02Z'}
routing_rules: 1 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'policy_id': 1, 'destination_id': 1, 'enabled':
1, 'priority': 0, 'created_at': '2026-07-06T07:54:02Z'}
delivery_attempts: 1 rows
    {'id': 1, 'content_item_id': 1, 'destination_id': 1, 'route_id': 1, 'status': 'delivered',
'attempt_count': 1, 'last_error': None, 'created_at': '2024-01-01T00:00:00Z', 'delivered_at':
'2024-01-01T00:00:00Z', 'next_retry_at': None}
content_items: 1 rows
    {'id': 1, 'account_id': 111, 'source_id': 1, 'provider': 'telegram', 'provider_message_id':
'5', 'content_type': 'video', 'text': None, 'link_url': None, 'media_ref': None, 'document_id':
None, 'duration_s': 60, 'width': 1920, 'height': 1080, 'size_bytes': 1000000, 'mime_type':
'video/mp4', 'dedup_key': 'tg:fp:1000000:60:1080:1920', 'matched': 1, 'seen_at':
'2024-01-01T00:00:00Z'}
```

28. assistant (2026-07-06T07:54:11.603Z)

Interesting results. Scenario A shows the delivery_attempts backfill is actually correct because of the additional `JOIN sources sc ON sc.provider_source_id = CAST(vd.channel_id AS TEXT)` which constrains to the right source. So no cartesian duplicate there ? the `sc` join disambiguates. Good, that concern is refuted.

Now the confirmed findings:
- **Scenario B**: numeric chat id `-1009999999999` gets `kind='channel'`, but ADR says numeric chat ids should be `'chat'` (ADR lists 'saved'|'channel'|'chat'). Minor.
- **Scenario C**: no filter_config ? 0 destinations/routing/delivery. Bounded by bootstrap always writing default config ? low severity.
- **is_output=0** for backfilled destination.

Now let me test the more important scenario: a video with the **same message_id in the SAME channel that appears with a document_id shared across channels** ? actually the real risk. Let

me test the delivery_attempts JOIN when a video was forwarded but the content_item got promoted to a `tg:doc:` dedup key that collapsed a duplicate. But more important: does the delivery_attempts JOIN drop rows? In scenario A both got attempts (2 rows for 2 videos). Good.

Let me test the critical partial-run idempotency: destinations inserted, then crash before routing_rules. Since the gate is `COUNT(destinations)>0`, a retry would skip everything. Let me simulate by manually inserting a destination then running migration 004 again on a DB stuck at version 3.

29. user (2026-07-06T07:54:30.563Z)

```
user_version after partial 004 crash: 3
destinations 1
routing_rules 0
delivery_attempts 0
final user_version: 4
```

30. assistant (2026-07-06T07:54:39.643Z)

This is a confirmed bug. The partial-run scenario:
1. Migration 004 runs, inserts destinations (step 1), then crashes before routing_rules (step 2) / delivery_attempts (step 3).
2. `user_version` stays at 3 (only bumped after `fn` completes).
3. On retry, migration 004 runs again but the gate `COUNT(destinations)>0 return` at line 681-683 sees the destination already exists and **returns early** ? skipping routing_rules and delivery_attempts entirely.
4. Result: `destinations=1, routing_rules=0, delivery_attempts=0`, and now `user_version=4` so 004 never runs again. **Permanently missing routing rules and delivery history.**

However ? is this realistic? The migration steps are all DML within a single connection, and `_apply_schema` only commits at the very end (line 217). Let me verify whether a crash mid-migration would actually leave the destinations committed. In `_apply_migrations`, there's no per-migration commit. So if the process crashes mid-004, the whole uncommitted transaction rolls back ? destinations would NOT persist. Let me verify there's no autocommit/implicit commit happening. aiosqlite/sqlite3 default `isolation_level` ? DDL like `ALTER TABLE`/`CREATE` and the connection's autocommit behavior matter. Let me check whether migration 003's `ALTER TABLE ... RENAME` causes an implicit commit.

31. user (2026-07-06T07:54:48.509Z)

```
isolation_level: ''
t count= 0
u ERR no such table: u
```

32. assistant (2026-07-06T07:54:59.765Z)

Important. With `isolation_level=''` (the aiosqlite/sqlite3 default), Python's sqlite3 module in legacy mode opens an implicit transaction before DML but **DDL statements are NOT auto-committed** in modern Python (this behavior changed in Python 3.6+). Here neither table persisted without an explicit commit ? closing rolled back. So closing the connection without commit rolls back everything.

But wait ? this matters for the crash scenario. If the process actually *crashes* (not clean close), the WAL/journal handles rollback on next open ? same effect: uncommitted work is discarded. So in a clean setup, migrations 001-004 all run within ONE uncommitted transaction and commit once at line 217. A crash mid-004 rolls back ALL of 001-004, user_version resets, and a full re-run happens cleanly.

So the partial-run bug is **NOT reachable via crash** under normal `init_db` flow, because there's no intermediate commit. The `COUNT(destinations)>0` gate's fragility only manifests if *something else* commits partway. Let me check: does any migration or the repo functions called during migration commit? Migration 003/004 use only `conn.execute` (no commit). But ? critical subtlety ? Python sqlite3 legacy `isolation_level=''` **auto-commits before DDL**? Let me re-verify: in repro3, the DDL did NOT commit the prior INSERT. So no.

But there's another path: the WAL checkpoint or a busy state. Actually, the real question is whether `ALTER TABLE RENAME` in migration 003 commits. It didn't in my test (CREATE TABLE didn't). Let me confirm ALTER specifically, and also confirm the full ladder is one transaction by checking if 004's early steps survive a simulated crash (kill without commit).

33. user (2026-07-06T07:55:07.506Z)

```
t count= 0
t2 ERR no such table: t2
```

34. assistant (2026-07-06T07:55:20.610Z)

Confirmed: even `ALTER TABLE RENAME` does not implicitly commit in this Python version. The entire migration ladder 001?004 runs as one uncommitted transaction, committed once at line 217. So a crash mid-004 rolls back everything cleanly, and the `COUNT(destinations)>0` gate fragility is **latent/not reachable** under the current `init_db` flow. I'll note it as a low-severity latent hygiene issue (the gate is not crash-safe if a future commit is introduced between migrations), but downgrade it since it can't fire now.

Now let me verify the remaining real concern: does the delivery_attempts backfill's unqualified `JOIN destinations d ON d.account_id = vd.user_id` produce a cartesian product if an account has **multiple destinations**? In the backfill only one destination is created per account, but let me test the robustness with a manually-added second destination ? this tests whether the SQL is correct in principle. Actually per the migration, only one destination is ever created (one filter_config row per account ? one destination). But if there were 2 (future), the join to `rr ON rr.destination_id = d.id` constrains it. Let me verify the routing_rules WHERE-subquery with multiple sources and confirm the `LIMIT 1` in the subquery is safe (one filter_config per account, so fine).

Let me now check one more real concern: the routing_rules backfill uses `d.target = (subquery)`. If `delivery_target` is a numeric id stored as int in JSON vs string. In models, `_coerce_delivery_target` coerces to str, but the migration reads raw legacy `config_json` which may have been written by the old code. Let me check what the old filter_config JSON stored ? was delivery_target always a string? Let me test a config where delivery_target is a JSON number.

35. user (2026-07-06T07:55:37.744Z)

```
Exit code 1

=== E: numeric JSON delivery_target ===
destinations: 1 ['-1009999999999']
routing_rules: 1 [1]
delivery_attempts: 1 [1]

=== F: 3 sources ===
destinations: 1 ['me']
routing_rules: 3 [1, 2, 3]
delivery_attempts: 2 [1, 2]
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro5.py", line 45, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
```

```
          ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro5.py", line 42, in main
    await run("G: malformed json", base+"g.db", 'not-json-at-all',
        videos=[(-1001,5,1)], channels=[(-1001,"a")])
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro5.py", line 24, in run
    await db.init_db(path)
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 196, in init_db
    await _apply_schema(conn)
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 216, in _apply_schema
    await _apply_migrations(conn)
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 324, in
_apply_migrations
    await fn(conn)
  File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 690, in
_migration_004_destinations_and_routing
    await conn.execute(
    ...<13 lines>...
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
           ^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.OperationalError: malformed JSON
```

36. assistant (2026-07-06T07:55:44.868Z)

Two important findings from this run:

Scenario G (malformed JSON): CONFIRMED bug. A `filter_config_deprecated.config_json` that is not valid JSON causes `json_extract(config_json, '$.delivery_target')` to raise `sqlite3.OperationalError: malformed JSON`, which **aborts the entire migration ladder** ? `init_db` throws, the app cannot start, and the whole upgrade fails. This is a **blocker-level** failure IF malformed JSON is reachable in real data.

But is malformed JSON reachable? The old `save_filter_config` always serialized a pydantic model to JSON, so stored configs should be valid JSON. However, `json_extract` also fails on valid-but-non-object JSON. More importantly, let me check reachability. The concern: is `config_json` guaranteed valid JSON object in production? Let me check how filter_config was written historically.

37. user (2026-07-06T07:55:46.427Z)

```
83:    config_json TEXT NOT NULL
132:    config_json TEXT NOT NULL,
476:                config_json TEXT NOT NULL
479:            "user_id, config_json",
622:        INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
624:        SELECT user_id, 'default', config_json, 1,
694:               COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
696:                 WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') = 'me'
718:                json_extract(fcd.config_json, '$.delivery_target'), 'me'
875:        config_json=row["config_json"],
1234:        """INSERT INTO policies (account_id, name, config_json, is_default,
1238:             config_json=excluded.config_json,
1244:            policy.config_json,
1672:    return FilterConfig.model_validate_json(policy.config_json)
```

```
1675:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1684:            config_json=cfg.model_dump_json(),
1772:    "save_filter_config",
```

38. user (2026-07-06T07:55:51.323Z)

```
1675    async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1676        """Deprecated shim: upsert the account's default :class:`Policy`."""
1677        existing = await get_default_policy(conn, user_id)
1678        await upsert_policy(
1679            conn,
1680            Policy(
1681                id=existing.id if existing is not None else None,
1682                account_id=user_id,
1683                name="default",
1684                config_json=cfg.model_dump_json(),
1685                is_default=True,
1686                created_at=(existing.created_at if existing is not None else _now()),
1687                updated_at=_now(),
1688            ),
1689        )
1690
1691
1692    # --- bulk helpers (used by the manager on startup) ---------------------------
1693
1694
```

39. assistant (2026-07-06T07:56:05.015Z)

So in production, `config_json` is always `FilterConfig.model_dump_json()` ? always valid JSON. So malformed JSON is not reachable through the normal write path. The malformed-JSON crash is therefore **latent** (only reachable via hand-edited/corrupted DB), so I'll downgrade it to medium/low. But note: migration 003 already copies `config_json` verbatim into policies without json_extract, so it never validated the JSON before. Migration 004 is the first to parse it with `json_extract`. This is a real robustness regression but low reachability.

Also note Scenario E: numeric JSON `delivery_target` works ? json_extract returns the number, SQLite coerces in comparison, and target stored as `-1009999999999`. But the routing_rules subquery compares `d.target = (json_extract...)`. `d.target` is stored TEXT `'-1009999999999'` while the subquery returns integer `-1009999999999`. In scenario E the routing rule WAS created (1 row), so SQLite's type affinity made it match. Good ? not a bug. But this is fragile. Let me confirm the comparison robustness isn't an issue ? it worked, so no finding.

Now let me confirm the remaining behavioral question: **the `is_output=0`** on the backfilled destination. ADR 0009 says `is_output` distinguishes output channels. A delivery destination is definitionally an output channel, so `is_output` arguably should be 1. Let me check how is_output is consumed anywhere (Track E, deferred). If nothing reads it, it's cosmetic. Let me grep.

40. user (2026-07-06T07:56:05.359Z)

```
src\tg_video_filter\models.py:237:    is_output: bool = Field(
src\tg_video_filter\db.py:150:    is_output   INTEGER NOT NULL DEFAULT 0,
src\tg_video_filter\db.py:692:        INSERT OR IGNORE INTO destinations (account_id, provider,
target, kind, is_output, created_at)
src\tg_video_filter\db.py:890:        is_output=bool(row["is_output"]),
src\tg_video_filter\db.py:1279:             (account_id, provider, target, kind, title,
is_output, created_at)
src\tg_video_filter\db.py:1284:             is_output=excluded.is_output""",
src\tg_video_filter\db.py:1291:            int(dest.is_output),
```

41. assistant (2026-07-06T07:56:10.778Z)

`is_output` is never read by any live logic (Track E deferred). So the `is_output=0` on a delivery destination is a latent semantic inaccuracy but has no behavioral impact yet ? low severity.

Now the `kind='channel'` for numeric chat ids: ADR says `'saved'|'channel'|'chat'`. A numeric id like `-1009999999999` is a channel/supergroup id in Telegram (the `-100` prefix denotes a channel/supergroup), so `'channel'` is arguably correct-ish, but a bare positive chat id (a user/small-group) would also be classified `'channel'`. The `kind` field is also never read by live logic (same grep pattern would show ? let me confirm kind isn't consumed).

42. user (2026-07-06T07:56:11.976Z)

```
src\tg_video_filter\db.py:692:        INSERT OR IGNORE INTO destinations (account_id, provider,
target, kind, is_output, created_at)
src\tg_video_filter\db.py:1114:            source.kind,
src\tg_video_filter\db.py:1282:             kind=excluded.kind,
src\tg_video_filter\db.py:1289:            dest.kind,
```

43. assistant (2026-07-06T07:56:17.428Z)

`kind` is stored/round-tripped but never used in any decision logic. So the numeric-id ? `'channel'` misclassification is cosmetic (low severity, latent).

Now let me re-examine the most consequential finding candidate: **the no-filter-config account (Scenario C)**. The prompt says bootstrap.py:75 always persists a default filter_config. Let me verify that, and also consider: a legacy DB from before that bootstrap behavior, or a user who deleted their config. Let me check bootstrap.

44. user (2026-07-06T07:56:18.531Z)

```
60          than a single conflated ``users`` row. The legacy ``User`` shape is
61          preserved as the return type so existing callers/tests are unaffected.
62          """
63          async with aiosqlite.connect(db_path) as conn:
64              await db._apply_schema(conn)
65              await db.upsert_account(conn, Account(id=user.id, telegram_id=user.id))
66              await db.upsert_telegram_session(
67                  conn,
68                  TelegramSession(
69                      account_id=user.id,
70                      phone=user.phone,
71                      session_path=f"user_{user.id}.session",
72                      enabled=user.enabled,
73                  ),
74              )
75              await db.save_filter_config(conn, user.id, DEFAULT_FILTER_CONFIG)
76              persisted = await db.get_user(conn, user.id)
77          assert persisted is not None  # just inserted
78          return persisted
79
80
81      async def bootstrap(  # pragma: no cover - interactive Telethon login (real connection +
human)
82          settings: Settings,
83          *,
84          phone_prompt: PhonePrompt = _default_phone_prompt,
```

45. assistant (2026-07-06T07:56:32.461Z)

Confirmed: bootstrap always writes a default filter_config for every onboarded account. So the no-config scenario only arises for an account that has legacy videos/channels but never completed bootstrap (structurally unusual ? you can't have forwarded videos without a running client which requires bootstrap). So Scenario C is essentially unreachable in practice ? low severity.

One more: the migration 003 backfills `policies` from `filter_config` (INSERT OR IGNORE), and migration 004's routing_rules JOIN requires `p.is_default = 1`. If an account has sources + destination but somehow no default policy, routing_rules would be silently skipped. But since destinations come from filter_config_deprecated and policies also come from filter_config, they're in lockstep. Not a separate bug.

Let me verify one final subtle thing about the delivery_attempts backfill: it requires a matching content_item via `provider_message_id`. But if migration 003's content_items backfill dropped a video (fingerprint collision keeping it at tg:msg ? no, it keeps ALL rows). content_items has one row per video always. And `ci.provider_message_id = CAST(vd.message_id AS TEXT)` + the `sc` join on channel_id fully disambiguates. So every forwarded video with a surviving content_item gets an attempt. Confirmed correct in scenario A/F (F: 2 forwarded videos ? 2 attempts).

I have enough. Let me also double check: in the delivery_attempts backfill, `route_id` is set to `rr.id`. And `created_at`/`delivered_at` use `vd.seen_at` ? that's the *seen* time, not the *forwarded* time. Legacy `videos` had no forwarded-at timestamp, so seen_at is the only available approximation. Minor semantic inaccuracy, acceptable ? not worth reporting as a defect.

Let me compile findings. The clean local gate already ran (451 passed), so these are edge cases. Let me finalize.

46. user (2026-07-06T07:57:10.150Z)

Structured output provided successfully